

核心概念		Core Concepts
StateGraph(State)	用 schema 建图	
node = 函数	输入 state,返回 partial	
edge	普通边:固定下一节点	
conditional_edge	按返回值路由	
START / END	入口 / 终点常量	
.add_node(name, fn)	注册节点	
.add_edge(a, b)	连接 a → b	
.add_conditional_edges	router 函数 + 映射	
.compile(checkpointer=)	编译为可执行 graph	
.invoke(input, cfg)	同步跑到 END	
.stream(...)	逐步产出	
最小示例		
g = StateGraph(State) g.add_node("a", fn_a) g.add_edge(START, "a") g.add_conditional_edges("a", route) app = g.compile(checkpointer=mem)		

Streaming		Streaming
.stream(input, cfg, stream_mode=)		
"values"	每步整个 state 快照	
"updates"	每步增量 {node: delta}	
"messages"	LLM token 级流式	
"debug"	含调度细节,排错用	
"custom"	node 内 emit 任意事件	
多 mode 列表	["updates","messages"] 一起拿	
异步 / 细粒度		
.astream(...)	async 版本	
astream_events(version=)	按 event 类型订阅	
on_chat_model_stream	逐 token	
on_tool_start / end	工具调用边界	
on_chain_start / end	node / 子链边界	
提示		
前端打字机用 messages,显示 step 进度用 updates		

State 设计		State
TypedDict	推荐的 state schema	
Pydantic BaseModel	需要校验时	
Annotated[T, reducer]	字段级合并策略	
add_messages	消息追加 + dedup by id	
operator.add	list / int 累加	
自定义 reducer	(old, new) → merged	
partial update	node 只返回改动字段	
无 reducer 字段	直接覆盖	
MessagesState	内置 messages 默认 schema	
典型 schema		
messages: Annotated[list, add_messages]		
step: int · plan: list[str] · scratchpad: str		
Input / Output schema		
支持给 StateGraph 单独传 input / output 子集,过滤外部可见字段		
提示		
state 字段越少越好,大对象用 store / 引用 ID		

子图 & Multi-Agent		Subgraphs
子图作为 node	compiled graph 当函数挂进父图	
共享 state key	同名字段自动透传	
独立 schema	不同字段需手动 map	
Send(node, state)	动态扇出 / map-reduce	
conditional_edges 返 [state, ...]	并行多份子任务	
Command(goto=, update=)	node 内同时跳转 + 改 state	
Command(graph=PARENT)	从子图跳回父图	
Supervisor 模式		
中央 supervisor node 决策 → 路由到具体 agent → 回到 supervisor;统一编排,易于增减成员		
Swarm 模式		
agent 之间通过 handoff 直接 Command(goto="other_agent") 互转,去中心化		
官方包		
langgraph-supervisor · langgraph-swarm 提供模板		

Checkpoint & 持久化		Persistence
MemorySaver()	进程内内存,开发用	
SqliteSaver	本地文件持久化	
PostgresSaver	生产 / 多实例	
AsyncPostgresSaver	异步版	
compile(checkpointer=)	编译时挂上	
thread_id	每个会话独立 history	
config={"configurable":}	透传 thread_id 等	
.get_state(cfg)	读当前快照	
.get_state_history(cfg)	所有 checkpoint	
.update_state(cfg, value)	人工改 state	
checkpoint_id	指定从某点恢复	
Time Travel		
把 history 中某个 checkpoint_id 传回 config,再 .invoke(None, cfg) 即可从该点继续或分叉		
Long-term store		
BaseStore 跨会话存档(用户偏好/记忆),区别于 thread 级 checkpoint		

Deep Agents		Deep Agents
设计灵感(Anthropic 风格)		
主 agent + planning + sub-agents + virtual filesystem,模仿 Claude Code / Deep Research		
create_deep_agent(...)一行造出深度 agent		
tools=[...] 领域工具列表		
instructions=... 主 agent system prompt		
subagents=[{...}] 隔离上下文的子 agent		
model=... 默认 Claude Sonnet		
write_todos / read_todos 内置 planning 工具		
write_file / read_file 虚拟 FS 存中间产物		
task(subagent, prompt)主 agent 派发给子 agent		
最小示例		
agent = create_deep_agent(tools=[search, fetch], instructions="你是研究员...", subagents=[{"name":"critic", "description":"审稿", "prompt":"挑毛病...", "tools":[search]},],)		
本质		
底层就是一张 LangGraph,可继续 .compile() 加 checkpoint / HITL		

HITL 人审介入		Human-in-the-Loop
interrupt(payload)	node 内暂停,payload 给前端	
Command(resume=val)	恢复并把 val 喂回 interrupt	
interrupt_before=[...]	编译时:进节点前停	
interrupt_after=[...]	编译时:出节点后停	
.update_state()	人工编辑 state 再继续	
.invoke(None, cfg)	从中断处继续	
三种 HITL 场景		
Approve → 工具调用前确认		
Edit → 修改 state / tool args 再继续		
Branch → time-travel 到旧节点重跑		
必备前提		
graph 必须 compile(checkpointer=...) 且调用时带 thread_id,否则不能恢复		
回包格式		
中断时 .invoke 返回含 __interrupt__ 的 state,前端读 payload 决定如何 resume		